

Spring Boot in the Cloud

Advanced Optimization Deep Dive

Patrick Baumgartner

42talents GmbH, Zürich, Switzerland

@patbaumgartner

patrick.baumgartner@42talents.com



Abstract

Spring Boot in the Cloud: Advanced Optimization Deep Dive

Spring Boot makes it easy to launch cloud-ready applications, but building truly lean and efficient cloud services requires going beyond the basics. In this three-hour deep dive, you'll immerse yourself in advanced Spring Boot techniques to optimize your applications for the cloud.

We will examine in depth how to reduce memory usage, improve startup times, and streamline your Spring Boot apps for modern cloud environments. Topics include Spring AOT, classpath exclusions, lazy beans, actuator configuration, custom JVM options, and additional state-of-the-art tools.

This session combines short presentations, live coding, and practical hands-on exercises. You will experiment with advanced configurations, observe real-time results, and troubleshoot performance challenges alongside the instructor. Real-world scenarios and best practices will help you immediately apply these techniques to your own projects.

Whether you want to fine-tune your CI/CD pipeline or push Spring Boot to its limits in production, you will leave with actionable skills, a collection of code samples, and a deeper understanding of cloud-native application optimization. Get ready for an in-depth, practical exploration of building lean Spring Boot applications for the cloud.

Spring Boot in the Cloud

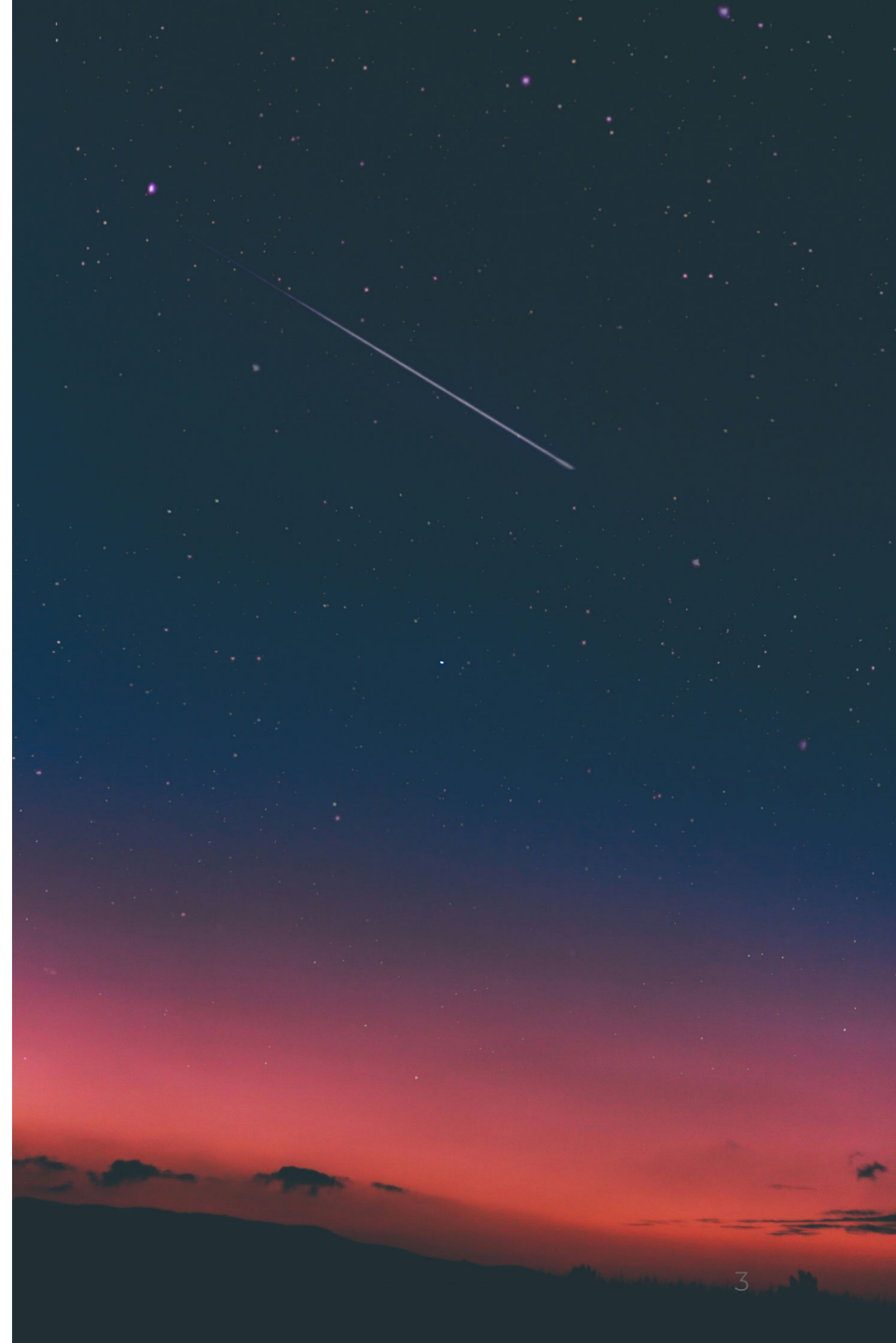
Advanced Optimization Deep Dive

Patrick Baumgartner

42talents GmbH, Zürich, Switzerland

@patbaumgartner

patrick.baumgartner@42talents.com



Spring Boot in the Cloud

Advanced Optimization Deep Dive

Patrick Baumgartner

42talents GmbH, Zürich, Switzerland

@patbaumgartner

patrick.baumgartner@42talents.com

bit.ly/48i5U9x



WARNING:

Numbers shown in this talk are **not** based on **real data** but **only estimates** and **assumptions** made by the **author** for **educational purposes** only.

Introduction



Patrick Baumgartner

Technical Agile Coach @ **42talents**

My focus is on the **development of software solutions *with* humans.**

Coaching, Architecture, Development, Reviews, and Training.

Lecturer @ **Zurich University of Applied Sciences ZHAW**

Co-Organizer of **Voxxed Days Zurich, JUG Switzerland**, Java Champion, Oracle ACE Pro Java ...

[@patbaumgartner](https://twitter.com/patbaumgartner)

What's the challenge?

Why does this matter?

I ❤️ Java 🤗 & Spring Boot 🌿



Introduction to Optimization Goals

Questions ...

Think about your current project.

- What would you optimise your application for?
- What are the key metrics?
- How would you measure them?

3' in pairs

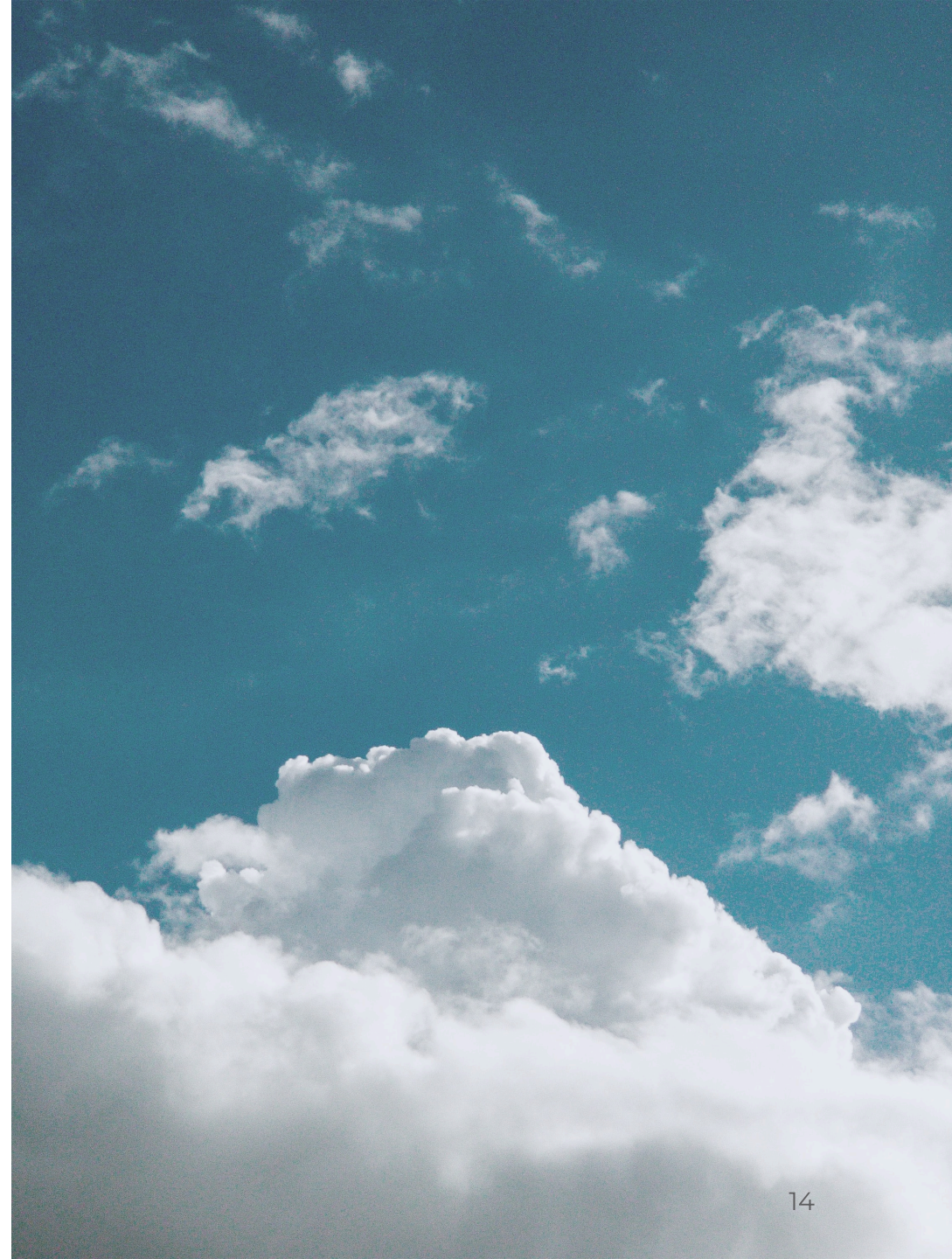
Measures

- Use Case
- Build Time
- Image Size
- Startup Time
- CPU Usage
- Memory Usage
- Throughput / Requests per Second
- Latency / Response Time

Requirements

When Deploying to a Cloud

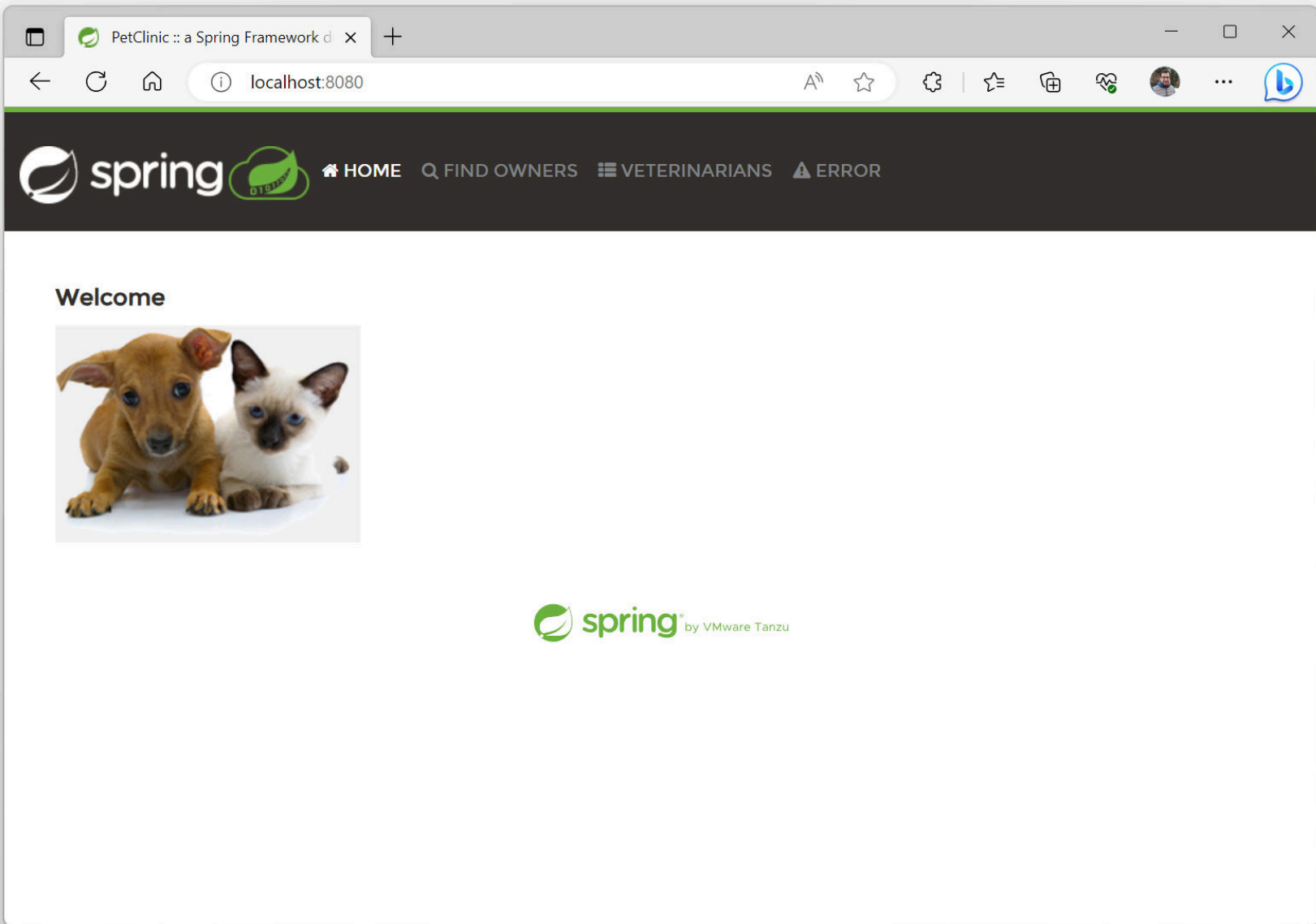
- How many vCPUs needs my application?
- How much RAM does it need?
- How much storage does it need?
- What technology stack should I use?
- What type of application do we build?



Agenda

Agenda

- Spring PetClinic & Baseline for comparison
- JVM Optimisations
- Spring Boot Optimisations
- Application Optimisations
- Other Runtimes
- Conclusions
- Some simple optimisations applied (OpenJDK examples)



PetClinic :: a Spring Framework d

×

+

←

↻

🏠

localhost:8080/vets.html

A

☆

⚙️

|

☆

🔒

🔒

🔒

🔒

...

🗣️

[🏠 HOME](#) [🔍 FIND OWNERS](#) [📋 VETERINARIANS](#) [⚠️ ERROR](#)

Veterinarians

Name	Specialties
James Carter	none
Helen Leary	radiology
Linda Douglas	dentistry surgery
Rafael Ortega	surgery
Henry Stevens	radiology

Pages: [1 [2](#)] 

 **spring**™ by VMware Tanzu

Spring Petclinic Community

- spring-framework-petclinic
- spring-petclinic-angular(js)
- spring-petclinic-rest
- spring-petclinic-graphql
- spring-petclinic-microservices
- spring-petclinic-data-jdbc
- spring-petclinic-cloud
- spring-petclinic-mustache
- spring-petclinic-kotlin
- spring-petclinic-reactive
- spring-petclinic-hilla
- spring-petclinic-angularjs
- spring-petclinic-vaadin-flow
- spring-petclinic-reactjs
- spring-petclinic-htmx
- spring-petclinic-istio
- ...

Projects on GitHub: <https://github.com/spring-petclinic>

NO!

The official Spring PetClinic!  
Based on Spring Boot, Caffeine,
Thymeleaf, Spring Data JPA, H2 and
Spring MVC ...

Project on GitHub: <https://github.com/spring-projects/spring-petclinic>

Limitations

Limitations

- Docker stats prints wrong values for memory usage, use docker top instead. (See: <https://quarkus.io/guides/performance-measure#measuring-memory-correctly-on-docker>)

```
docker top <CONTAINER ID> -o pid,rss,args
```

- Thermal throttling of your notebook CPU may affect your results

Benchmark Steps

Build with Maven

Set the Java version and build the OCI container image.

```
sdk use java 17.0.18-librca
java -version

build_time=$(./mvnw clean spring-boot:build-image \
  | grep "Total time:" \
  | awk '{print $4 $5}')

echo -e "\033[0;32mBuild time: $build_time\033[0m"

image_size=$(docker images \
  | grep "spring-petclinic" \
  | awk '{print $7}')

echo -e "\033[0;32mImage Size: $image_size\033[0m"
```

Container Startup Time

Run the container with limited memory and measure the startup time.

```
docker container run -d --rm --memory="1g" -p 8080:8080 -t spring-petclinic:4.0.0-SNAPSHOT

# Get the container ID of the running container
container_id=$(docker ps -q)

startup_time=$(docker logs "$container_id"
| grep "Started" \
| awk '{print $14 $15}')

echo -e "\033[0;32mStartup time: $startup_time\033[0m"
```

Memory Usage after Startup

Extract the memory usage of the running container. Convert rss to MB.

```
memory=$(docker top $(docker ps -lq) -o pid,rss,args \  
| tail -n +2 \  
| awk '{ $2=int($2/1024)"MB"; }{ print $2; }')  
  
echo -e "\033[0;32mStartup memory usage: $memory\033[0m"
```


Warm-up Application

Warm up the application with for 60s, json output and no UI.

```
oha --no-tui -z60s --disable-keepalive -j "http://localhost:8080/owners?page=2"
```

Load-Test Application

Load test during 60s, json output and no UI.

```
throughput=$(oha --no-tui -z60s --disable-keepalive -j "http://localhost:8080/owners?page=2" 2>&1 \  
| tee $filename \  
| jq '.summary.requestsPerSec')  
  
echo -e "\033[0;32mAvg Throughput: $throughput Requests/sec\033[0m"
```

Memory Usage after Load-Test

Extract the memory usage of the running container. Convert rss to MB.

```
memory=$(docker top $(docker ps -lq) -o pid,rss,args \  
| tail -n +2 \  
| awk '{ $2=int($2/1024)"MB"; } { print $2; }')  
  
echo -e "\033[0;32mEnd memory usage: $memory\033[0m"
```

Optimisation Experiments

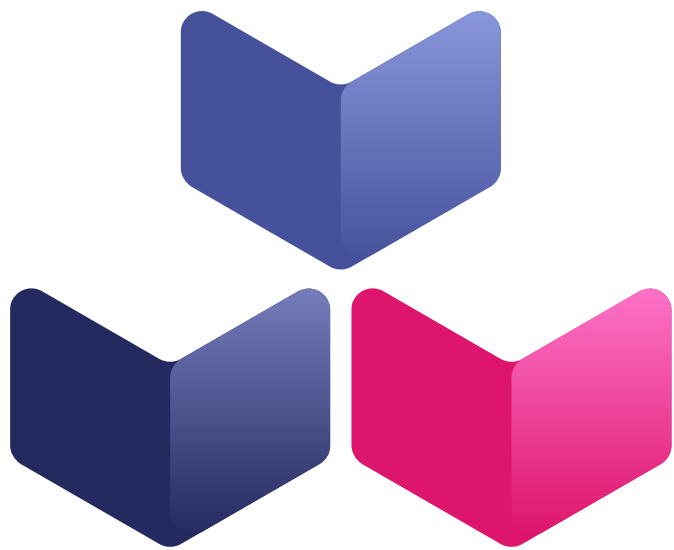
Baseline

Technology Stack

- OCI Container built with Buildpacks
- Java JDK 17 LTS
- Spring Boot 4.0
- Testcontainers
- DB migration using SQL scripts

Examination

- Build time
- Startup time
- Resource usage
- Container image size
- Throughput



Buildpacks.io



paketo
buildpacks



**Your app,
in your favorite language,
ready to run in the cloud**



GraalVM



NGINX

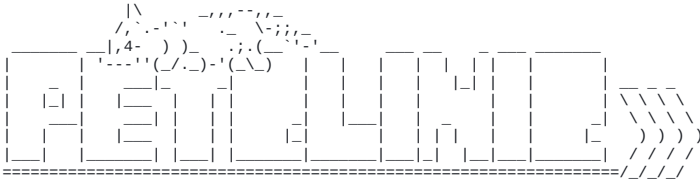


python

```

Calculating JVM memory based on 22958184K available memory
For more information on this calculation, see https://paketo.io/docs/reference/java-reference/#memory-calculator
Calculated JVM Memory Configuration: -XX:MaxDirectMemorySize=10M -Xmx22316444K -XX:MaxMetaspaceSize=129739K -XX:ReservedCodeCacheSize=240M -Xss1M
(Total Memory: 22958184K, Thread Count: 250, Loaded Class Count: 20492, Headroom: 0%)
Enabling Java Native Memory Tracking
Adding 146 container CA certificates to JVM truststore
Spring Cloud Bindings Enabled
Picked up JAVA_TOOL_OPTIONS: -Djava.security.properties=/layers/paketo-buildpacks_bellsoft-liberica/java-security-properties/java-security.properties
-XX:+ExitOnOutOfMemoryError -XX:MaxDirectMemorySize=10M -Xmx22316444K -XX:MaxMetaspaceSize=129739K -XX:ReservedCodeCacheSize=240M -Xss1M
-XX:+UnlockDiagnosticVMOptions -XX:NativeMemoryTracking=summary -XX:+PrintNMTStatistics -Dorg.springframework.cloud.bindings.boot.enable=true

```



```
:: Built with Spring Boot :: 4.0.5
```

```

2026-03-29T18:00:07.524Z INFO 1 --- [main] o.s.s.petclinic.PetClinicApplication : Starting PetClinicApplication v4.0.0-SNAPSHOT using Java 17.0.18
with PID 1 (/workspace/BOOT-INF/classes started by cnb in /workspace)
2026-03-29T18:00:07.528Z INFO 1 --- [main] o.s.s.petclinic.PetClinicApplication : No active profile set, falling back to 1 default profile: "default"
2026-03-29T18:00:08.469Z INFO 1 --- [main] s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-03-29T18:00:08.527Z INFO 1 --- [main] s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 47 ms. Found 3 JPA repository interfaces.
2026-03-29T18:00:09.365Z INFO 1 --- [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-03-29T18:00:09.380Z INFO 1 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-03-29T18:00:09.381Z INFO 1 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.18]
2026-03-29T18:00:09.440Z INFO 1 --- [main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 1860 ms
2026-03-29T18:00:09.733Z INFO 1 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-03-29T18:00:09.937Z INFO 1 --- [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection conn0: url=jdbc:h2:mem:8a2f72dc-247f-4908-98aa-3a8d6d1fd45a user=SA
2026-03-29T18:00:09.940Z INFO 1 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-03-29T18:00:10.124Z INFO 1 --- [main] org.hibernate.orm.jpa : HHH000540: Processing PersistenceUnitInfo [name: default]
2026-03-29T18:00:10.182Z INFO 1 --- [main] org.hibernate.orm.core : HHH000001: Hibernate ORM core version 7.2.4.Final
2026-03-29T18:00:08.145Z INFO 1 --- [main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-03-29T18:00:08.205Z INFO 1 --- [main] org.hibernate.orm.connections.pooling : HHH10001005: Database info:
Database JDBC URL [jdbc:h2:mem:8a2f72dc-247f-4908-98aa-3a8d6d1fd45a]
Database driver: H2 JDBC Driver
Database dialect: H2Dialect
Database version: 2.4.240
Default catalog/schema: 8A2F72DC-247F-4908-98AA-3A8D6D1FD45A/PUBLIC
Autocommit mode: undefined/unknown
Isolation level: READ_COMMITTED [default READ_COMMITTED]
JDBC fetch size: 100
Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-03-29T18:00:09.147Z INFO 1 --- [main] org.hibernate.orm.core : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-03-29T18:00:09.153Z INFO 1 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-03-29T18:00:09.250Z INFO 1 --- [main] o.s.d.j.r.query.QueryEnhancerFactories : Hibernate is in classpath; If applicable, HQL parser will be used.
2026-03-29T18:00:10.577Z INFO 1 --- [main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 13 endpoints beneath base path '/actuator'
2026-03-29T18:00:10.688Z INFO 1 --- [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-03-29T18:00:10.701Z INFO 1 --- [main] o.s.s.petclinic.PetClinicApplication : Started PetClinicApplication in 3.621 seconds (process running for 6.498)

```

1000x better than your regular
Dockerfile  ...

... more **Secure**  and maintained by the
Buildpacks community.

See also: <https://buildpacks.io/> and <https://www.cncf.io/projects/buildpacks/>

Friends don't
let friends write
Dockerfiles!

@starbuxman



Benchmarks

Benchmarks

- Build
 - Maven build time
 - Artifact / Container Image size
- Startup
 - Startup time
 - Memory usage
- Throughput & Latency
 - `oha --no-tui -z60s --disable-keepalive <URL>`
 - 60s warmup, 60s measurement
 - Docker container with 2 vCPU and 1 GB RAM

No Optimizing - Baseline JDK 17

- Spring PetClinic (no adjustments)
- Bellsoft Liberica JDK 17.0.18
- Java Memory Calculator

```
sdk use java 17.0.18-librca  
mvn spring-boot:build-image  
docker run -p 8080:8080 -t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms

No Optimizing - Baseline JDK 21

- Spring PetClinic (JDK 21 adjustments)
- Bellsoft Liberica JDK 21.0.10
- Java Memory Calculator

```
sdk use java 21.0.10-librca
```

```
mvn -Djava.version=21 spring-boot:build-image \  
    -Dspring-boot.build-image.imageName=spring-petclinic:4.0.0-SNAPSHOT-jdk21
```

```
docker run -p 8080:8080 -t spring-petclinic:4.0.0-SNAPSHOT-jdk21
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 21	64s	307MB	4.943s	273MB	257/s	383MB	184ms	292ms	404ms	702ms	994ms

No Optimizing - Baseline JDK 25

- Spring PetClinic (JDK 25 adjustments)
- Bellsoft Liberica JDK 25.0.2
- Java Memory Calculator

```
sdk use java 25.0.2-librca
```

```
mvn -Djava.version=25 spring-boot:build-image \  
    -Dspring-boot.build-image.imageName=spring-petclinic:4.0.0-SNAPSHOT-jdk25
```

```
docker run -p 8080:8080 -t spring-petclinic:4.0.0-SNAPSHOT-jdk25
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 25	66s	376MB	4.887s	270MB	227/s	438MB	194ms	305ms	464ms	771ms	1040ms

No Optimizing - Baseline JDK 26

- Spring PetClinic (JDK 26 adjustments)
- Bellsoft Liberica JDK 26
- Java Memory Calculator

```
sdk use java 26-librca
```

```
mvn -Djava.version=26 spring-boot:build-image \  
    -Dspring-boot.build-image.imageName=spring-petclinic:4.0.0-SNAPSHOT-jdk26
```

```
docker run -p 8080:8080 -t spring-petclinic:4.0.0-SNAPSHOT-jdk26
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 26	66s	382MB	4.939s	269MB	256/s	395MB	184ms	293ms	404ms	694ms	908ms

JVM Optimisations

-noverify

The verifier is turned off because some of the bytecode rewriting stretches the meaning of some bytecodes - in a way that doesn't bother the JVM, but does bother the verifier.

Warning: The `-Xverify:none` and `-noverify` options are deprecated in JDK 13 and are likely to be removed in a future release.

```
docker run -p 8080:8080 -e "JAVA_TOOL_OPTIONS=-noverify" \  
-t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	-	-	4.422s	243MB	220/s	355MB	198ms	307ms	468ms	791ms	1103ms
Java 21	-	-	4.193s	252MB	235/s	372MB	192ms	300ms	440ms	782ms	1010ms
Java 25	-	-	4.282s	252MB	234/s	417MB	192ms	301ms	435ms	727ms	971ms

-XX:+UseCompactObjectHeaders

The Compact Object Headers (JEP 384) feature reduces the memory footprint of Java objects by using a more compact representation for object headers. This can lead to improved cache performance and reduced memory usage.

```
docker run -p 8080:8080 -e "JAVA_TOOL_OPTIONS=-XX:+UseCompactObjectHeaders" \
-t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
 Java 25	66s	376MB	4.887s	270MB	227/s	438MB	194ms	305ms	464ms	771ms	1040ms
Java 25	-	-	5.028s	264MB	194/s	441MB	209ms	366ms	504ms	828ms	1082ms

-XX:TieredStopAtLevel=1

Tiered compilation is enabled by default (since Java 8). Unless explicitly specified, the JVM decides which JIT compiler to use based on our CPU. For multi-core processors or 64-bit VMs, the JVM will select C2.

To disable C2 and use only the C1 compiler with no profiling overhead, we can use the `-XX:TieredStopAtLevel=1` parameter.

```
docker run -p 8080:8080 -e "JAVA_TOOL_OPTIONS=-XX:TieredStopAtLevel=1" \
-t spring-petclinic:4.0.0-SNAPSHOT
```

It will slow down the JIT later at the expense of the saved startup time!

	Build	Image	Startup	Initial RAM	T...	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	-	-	3.895s	222MB	152/s	296MB	308ms	406ms	503ms	699ms	814ms
Java 21	-	-	4.058s	232MB	146/s	303MB	316ms	417ms	517ms	710ms	890ms
Java 25	-	-	3.75s	227MB	174/s	385MB	288ms	394ms	499ms	703ms	892ms

-XX:+UseParallelGC

The Parallel Garbage Collector (Parallel GC) is a high-throughput garbage collector that is designed to take advantage of multiple processors. It performs minor garbage collections in parallel, which can lead to shorter pause times and improved application performance.

```
docker run -p 8080:8080 -e "JAVA_TOOL_OPTIONS=-XX:+UseParallelGC" \
-t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	T...	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 21	-	-	4.886s	349MB	249/s	474MB	190ms	294ms	407ms	704ms	995ms
Java 25	-	-	4.728s	348MB	242/s	472MB	190ms	299ms	432ms	720ms	931ms
Java 26	-	-	4.709s	305MB	253/s	452MB	187ms	294ms	403ms	691ms	897ms

-XX:+UseZGC -XX:+ZGenerational

The Z Garbage Collector (ZGC) is a scalable, low-latency garbage collector. ZGC performs all expensive work concurrently without stopping application threads for more than 10ms, making it suitable for applications that require low latency and/or use a very large heap (multi-terabyte).

```
docker run -p 8080:8080 -e "JAVA_TOOL_OPTIONS=-XX:+UseZGC -XX:+ZGenerational" \
-t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	T...	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 21	-	-	5.17s	524MB	225/s	673MB	195ms	306ms	490ms	803ms	1157ms
Java 25	-	-	5.237s	495MB	213/s	655MB	199ms	332ms	495ms	826ms	1099ms

See also: <https://wiki.openjdk.org/display/zgc/Main>

VM Options Explorer

<https://chriswhocodes.com>

VM Options Explorer - OpenJDK11

<https://chriswhocodes.com>

Byte-Me FullJEP JEPMap JEPSearch hsdisk JITWatch JaCoLine VM Options Explorer VM Intrinsic Explorer GC Explorer Optimizing Java Thank You!

VM Options Explorer - OpenJDK11 HotSpot

[OpenJDK HotSpot](#)

[Options added/removed between JDKs](#)

OpenJDK options also hosted on [foojay.io](#)

6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26

[Alibaba Dragonwell](#)

8	11	17	21

[Amazon Corretto](#)

8	11	17	19	20	21	22	24

[Azul Systems](#)

Platform Prime								Zulu									
8	11	13	15	17	19	8	11	13	15	16	17	18	19	20	21	22	24

[BellSoft Liberica](#)

8	11	17	18	19	20	21	22

[Eclipse Temurin](#)

8	11	17	18	19	20	21	22

[JDK-based GraalVM](#)

17		21		22		24	
JDK	Native	JDK	Native	JDK	Native	JDK	Native

[JetBrains Runtime](#)

11	17	21

[Microsoft](#)

11	16	17	21

[OpenJ9](#)

[OpenJ9](#)

[Oracle](#)

6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24

[SAP SapMachine](#)

11	17	19	20	21

Search OpenJDK11 HotSpot Options:

Name	Since	Deprecated	Type	OS	CPU	Component	Default	Availability	Description	Defined in
Show All	Deprecated	Show All	Show All	Show All	Show All	Show All	Show All	Show All		

Spring Boot Optimisations

Lazy Spring Beans (1)

Configure lazy initialisation throughout your application. A Spring Boot property makes all beans lazy by default, initialising them only when needed. You can use `@Lazy` to override this behaviour, e.g. `@Lazy(false)`.

```
docker run -p 8080:8080 \
  -e spring.main.lazy-initialization=true \
  -e spring.data.jpa.repositories.bootstrap-mode=lazy \
  -t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	-	-	4.501s	239MB	234/s	356MB	192ms	301ms	438ms	740ms	1002ms

Lazy Spring Beans (2)

Pros

- Faster startup useful in cloud environments
- Application startup is a CPU-intensive task. Spreads load over time

Cons

- Initial requests may take longer
- Class loading problems and misconfigurations not detected at startup
- Beans creation errors only be found when the bean is loaded

Fixing Spring Boot Config Location

Determine the location of the Spring Boot configuration file(s).
Considered in the following order (`application.properties` and YAML variants)

- Application properties packaged in your jar
- Profile-specific application properties packaged within your jar
- Application properties outside your packaged jar
- Profile-specific application properties outside your packaged jar

```
docker run -p 8080:8080 -e spring.config.location=classpath:application.properties \
  -t spring-note:linux:4.0.0 SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	-	-	5.099s	257MB	232/s	358MB	192ms	302ms	456ms	758ms	1013ms

No Spring Boot Actuators

Don't use actuators if you can afford not to 😊.

- Number of Spring Beans
 - Spring Pet Clinic with actuators: 457
 - Spring Pet Clinic without actuators: 282 🔥

```
sdk use java 17.0.18-librca
mvn spring-boot:build-image
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
🕒 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	64s	277MB	4.466s	246MB	249/s	361MB	185ms	295ms	406ms	701ms	972ms
Java 21	63s	305MB	4.316s	252MB	265/s	360MB	176ms	289ms	400ms	694ms	985ms
Java 25	64s	373MB	4.442s	250MB	256/s	455MB	182ms	292ms	405ms	693ms	926ms

Ahead-of-Time Processing (AOT) (1)

Spring AOT is a process that analyses your application at build time and generates an optimised version of it.

As the `BeanFactory` is fully prepared at build time, conditions are also evaluated. E.g. in `Configurations`, `Component- & Entity-Scan`, `@Profile`, `@Conditional`, etc.

Spring AOT serves as a replacement for `spring-context-indexer` since Spring Framework 6.1 and Spring Boot 3.2.

Ahead-of-Time Processing (AOT) (2)

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <env>
        <BP_SPRING_AOT_ENABLED>true</BP_SPRING_AOT_ENABLED>
      </env>
    </image>
  </configuration>
  <executions>
    <execution>
      <id>process-aot</id>
      <goals>
        <goal>process-aot</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

Ahead-of-Time Processing (AOT) (3)

We create a new container image with the AOT-processed application. The Buildpack enables the property `spring.aot.enabled=true`

```
sdk use java 17.0.18-librca  
  
mvn spring-boot:build-image  
  
docker run -p 8080:8080 -e spring.aot.enabled=true \  
-t spring-petclinic-aot:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	71s	281MB	4.095s	242MB	227/s	352MB	195ms	303ms	479ms	786ms	1031ms
Java 21	71s	309MB	3.912s	245MB	251/s	368MB	187ms	294ms	407ms	706ms	984ms
Java 25	70s	377MB	4.03s	237MB	245/s	541MB	189ms	296ms	409ms	700ms	909ms

Disabling JMX

JMX is `spring.jmx.enabled=false` by default in Spring Boot since 2.2.0 and later. Setting `BPL_JMX_ENABLED=true` and `BPL_JMX_PORT=9999` on the container will add the following arguments to the `java` command.

```
-Djava.rmi.server.hostname=127.0.0.1  
-Dcom.sun.management.jmxremote.authenticate=false  
-Dcom.sun.management.jmxremote.ssl=false  
-Dcom.sun.management.jmxremote.port=9999  
-Dcom.sun.management.jmxremote.rmi.port=9999
```

```
docker run -p 8080:8080 -p 9999:9999 \  
  -e BPL_JMX_ENABLED=false \  
  -e BPL_JMX_PORT=9999 \  
  -e spring.jmx.enabled=false \  
  -t spring-petclinic:4.0.0-SNAPSHOT
```

I ❤️
Spring Boot 🌿 &
Buildpacks 🚀

Application Optimisations

Dependency Cleanup (2)

DepClean detects all unused dependencies declared in the `pom.xml` file of a project and creates a `pom-debloated.xml`. The generated report shows possible unused dependencies.

```
<plugin>
  <groupId>se.kth.castor</groupId>
  <artifactId>depclean-maven-plugin</artifactId>
  <version>2.2.0-SNAPSHOT</version>
  <executions>
    <execution>
      <goals>
        <goal>depclean</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

```
mvn se.kth.castor:depclean-maven-plugin:2.2.0-SNAPSHOT:depclean -DfailIfUnusedDirect=true -DignoreScopes=provided,test,runtime,system,import
```

D E P C L E A N A N A L Y S I S R E S U L T S

USED DIRECT DEPENDENCIES [10]:

com.h2database:h2:2.4.240:runtime (2 MB)
com.mysql:mysql-connector-j:9.6.0:runtime (2 MB)
org.postgresql:postgresql:42.7.10:runtime (1 MB)
com.github.ben-manes.caffeine:caffeine:3.2.3:runtime (996 KB)
jakarta.xml.bind:jakarta.xml.bind-api:4.0.4:compile (128 KB)
org.springframework.boot:spring-boot-testcontainers:4.0.5:test (74 KB)

...

USED TRANSITIVE DEPENDENCIES [107]:

org.testcontainers:testcontainers:2.0.3:test (16 MB)
org.hibernate.orm:hibernate-core:7.2.4.Final:compile (13 MB)
net.bytebuddy:byte-buddy:1.17.8:runtime (8 MB)
org.apache.tomcat.embed:tomcat-embed-core:11.0.18:compile (3 MB)
com.github.docker-java:docker-java-transport-zero-dep:3.7.0:test (2 MB)
org.springframework:spring-web:7.0.5:compile (2 MB)

...

USED INHERITED DIRECT DEPENDENCIES [0]:

USED INHERITED TRANSITIVE DEPENDENCIES [0]:

POTENTIALLY UNUSED DIRECT DEPENDENCIES [14]:

org.webjars.npm:bootstrap:5.3.8:runtime (1 MB)
org.webjars.npm:font-awesome:4.7.0:runtime (665 KB)
org.springframework.boot:spring-boot-devtools:4.0.5:compile (208 KB)
org.springframework.boot:spring-boot-docker-compose:4.0.5:test (123 KB)
org.springframework.boot:spring-boot-starter-actuator:4.0.5:compile (4 KB)

...

POTENTIALLY UNUSED TRANSITIVE DEPENDENCIES [34]:

commons-codec:commons-codec:1.19.0:test (365 KB)
org.springframework.boot:spring-boot-actuator:4.0.5:compile (348 KB)
org.yaml:snakeyaml:2.5:compile (332 KB)
org.attoparser:attoparser:2.0.7.RELEASE:compile (240 KB)
org.springframework.boot:spring-boot-micrometer-metrics:4.0.5:compile (235 KB)
org.thymeleaf:thymeleaf-spring6:3.1.3.RELEASE:compile (184 KB)
org.hdrhistogram:HdrHistogram:2.2.2:runtime (173 KB)
org.unbescape:unbescape:1.1.6.RELEASE:compile (169 KB)
org.springframework.boot:spring-boot-web-server:4.0.5:compile (165 KB)
org.springframework.boot:spring-boot-health:4.0.5:compile (150 KB)

...

POTENTIALLY UNUSED INHERITED DIRECT DEPENDENCIES [0]:

POTENTIALLY UNUSED INHERITED TRANSITIVE DEPENDENCIES [0]:

[INFO] Analysis done in 0min 27s

Dependency Cleanup (2)

But there are some challenges:

- Component & Entity Scanning through Classpath Scanning
- Spring Boot uses `META-INF/spring-boot/org.springframework.boot.autoconfigure.AutoConfiguration.imports`
- Spring XML Configuration and `web.xml`

```
sdk use java 17.0.18-librca
```

```
mvn spring-boot:build-image
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	37s	270MB	4.278s	245MB	256/s	349MB	177ms	292ms	402ms	700ms	969ms
Java 21	36s	297MB	4.024s	245MB	248/s	372MB	179ms	296ms	409ms	735ms	1002ms
Java 25	37s	366MB	4.086s	240MB	277/s	437MB	142ms	280ms	393ms	667ms	901ms

More Application Optimisations

- Application Optimisations
- Better Connection Pooling
- Improved Data Caching
- Database Queries Optimisations
- Batch Processing Optimisations
- Reduce Garbage Collection
- Monitor and Tune JVM Settings
- Reduce Overall Memory Footprint
- Async Logging

Even More JVM Optimisations

JLink (1)

`jlink` assembles and optimises a set of modules and their dependencies into a custom runtime image for your application.

```
$ jlink \  
  --add-modules java.base, ... \  
  --strip-debug \  
  --no-man-pages \  
  --no-header-files \  
  --compress=2 \  
  --output /javaruntime
```

```
$ /javaruntime/bin/java HelloWorld  
Hello, World!
```

JLink (2)

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <env>
        <BP_JVM_JLINK_ENABLED>true</BP_JVM_JLINK_ENABLED>
      </env>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	74s	191MB	5.003s	264MB	238/s	374MB	190ms	299ms	433ms	735ms	1002ms
Java 21	74s	199MB	4.943s	284MB	258/s	388MB	184ms	291ms	404ms	700ms	986ms
Java 25	72s	201MB	4.928s	282MB	268/s	386MB	178ms	286ms	395ms	652ms	897ms

Java Dependency Analysis Tool

`jdeps` is a Java command-line tool that analyzes class and jar files to list the package-level or class-level dependencies, helping developers understand module dependencies and improve code modularity.

```
jar xvf target/spring-petclinic-4.0.0-SNAPSHOT.jar

jdeps --ignore-missing-deps -q \
      --recursive \
      --multi-release 17 \
      --print-module-deps \
      --class-path 'BOOT-INF/lib/*' \
      target/spring-petclinic-4.0.0-SNAPSHOT.jar
```

```
java.base,java.compiler,java.desktop,java.instrument,java.net.http,java.prefs,java.rmi,java.scripting,java.security.jgss,java.sql.rowset,jdk.jfr,jdk.management,jdk.unsupported
```

JLink (3)

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <env>
        <BP_JVM_JLINK_ENABLED>true</BP_JVM_JLINK_ENABLED>
        <BP_JVM_JLINK_ARGS>--add-modules java.base,java.compiler,java.desktop,java.instrument,
        java.net.http,java.prefs,java.rmi,java.scripting,java.security.jgss,java.sql.rowset,
        jdk.jfr,jdk.management,jdk.unsupported --compress=2 --no-header-files --no-man-pages
        --strip-debug</BP_JVM_JLINK_ARGS>
      </env>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	72s	179MB	5.13s	266MB	232/s	368MB	194ms	302ms	443ms	763ms	1041ms
Java 21	71s	187MB	4.964s	283MB	256/s	390MB	185ms	291ms	403ms	701ms	990ms
Java 25	70s	189MB	4.949s	277MB	262/s	378MB	181ms	289ms	399ms	685ms	903ms

App CDS (1)

Class Data Sharing (CDS) is a JVM feature that can help reduce the startup time and memory footprint of Java applications. It allows classes to be pre-processed into a shared archive file that can be memory-mapped at runtime.

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <env>
        <BP_JVM_CDS_ENABLED>true</BP_JVM_CDS_ENABLED>
      </env>
    </image>
  </configuration>
</plugin>
```

App CDS (2)

To create the archive, two additional JVM flags must be specified:

- `-XX:ArchiveClassesAtExit=application.jsa` : creates the CDS archive on exit
- `-Dspring.context.exit=onRefresh` : starts and then immediately exits

Once the archive is available, the JVM can be started with the additional flag:

- `-XX:SharedArchiveFile=application.jsa` : to use it

```
sdk use java 17.0.18-librca
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	80s	450MB	2.64s	236MB	257/s	344MB	179ms	291ms	402ms	703ms	993ms
Java 21	79s	477MB	2.635s	246MB	280/s	364MB	121ms	280ms	392ms	683ms	915ms

I ❤️

Spring Boot 🌿 &
Buildpacks 🚀

AOT Caching

Project Leyden is an OpenJDK project that aims to improve the startup time and reduce the memory footprint of Java applications by introducing AOT (Ahead-of-Time) compilation and class data sharing (CDS) enhancements.

Training the AOT cache requires two additional JVM flags:

`-XX:AOTCacheOutput=app.aot -Dspring.context.exit=onRefresh`

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 25	83s	570MB	1.98s	240MB	301/s	427MB	117ms	218ms	379ms	597ms	807ms

Other Runtimes



Unleash the power of Java

Optimized to run Java™ applications cost-effectively in the cloud, Eclipse OpenJ9™ is a fast and efficient JVM that delivers power and performance when you need it most.

Optimized for the Cloud, for microservices and monoliths too!

Faster Startup

Faster Ramp-up, when deployed to cloud

Smaller

Our Story

Eclipse OpenJ9

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <buildpacks>
        <buildpack>paketobuildpacks/eclipse-openj9:latest</buildpack>
        <!-- Used to inherit all the other Java buildpacks -->
        <buildpack>urn:cnb:builder:paketo-buildpacks/java</buildpack>
      </buildpacks>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	73s	266MB	5.114s	180MB	97/s	277MB	499ms	621ms	864ms	1233ms	1556ms
Java 21	71s	284MB	5.373s	185MB	97/s	278MB	500ms	663ms	860ms	1183ms	1506ms
Java 25	71s	283MB	5.756s	185MB	89/s	271MB	575ms	694ms	864ms	1159ms	1366ms

GraalVM CE

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <buildpacks>
        <buildpack>paketobuildpacks/graalvm:latest</buildpack>
        <!-- Used to inherit all the other Java buildpacks -->
        <buildpack>urn:cnb:builder:paketo-buildpacks/java</buildpack>
      </buildpacks>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	83s	678MB	5.136s	292MB	185/s	385MB	213ms	394ms	582ms	914ms	1272ms
Java 21	81s	663MB	5.118s	311MB	190/s	408MB	204ms	391ms	539ms	934ms	1268ms
Java 25	82s	851MB	5.167s	322MB	210/s	420MB	200ms	318ms	499ms	805ms	1112ms

Oracle JDK

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <buildpacks>
        <buildpack>paketobuildpacks/oracle:latest</buildpack>
        <!-- Used to inherit all the other Java buildpacks -->
        <buildpack>urn:cnb:builder:paketo-buildpacks/java</buildpack>
      </buildpacks>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 21	70s	448MB	4.908s	274MB	240/s	382MB	190ms	299ms	431ms	746ms	1015ms
Java 25	70s	492MB	4.96s	273MB	259/s	446MB	183ms	290ms	400ms	685ms	893ms

Azul Zulu

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <buildpacks>
        <buildpack>paketobuildpacks/azul-zulu:latest</buildpack>
        <!-- Used to inherit all the other Java buildpacks -->
        <buildpack>urn:cnb:builder:paketo-buildpacks/java</buildpack>
      </buildpacks>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	66s	243MB	5.115s	257MB	233/s	368MB	192ms	301ms	442ms	788ms	1039ms
Java 21	66s	267MB	4.855s	278MB	259/s	379MB	183ms	291ms	402ms	701ms	988ms
Java 25	65s	301MB	4.998s	272MB	276/s	385MB	158ms	284ms	394ms	642ms	899ms

Other Buildpack Builders

Bellsoft Buildpack Builder

Bellsoft provides an optimised builder for Spring Boot applications. It uses the Bellsoft Alpaquita, Liberica JDK and the musl C library. A glibc version is also available.

```
<image>  
  <builder>bellsoft/buildpacks.builder:musl</builder>  
  <!--builder>bellsoft/buildpacks.builder:glibc</builder-->  
</image>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17 (m)	65s	183MB	4.951s	288MB	258/s	387MB	184ms	292ms	405ms	707ms	995ms
Java 21 (m)	63s	191MB	4.69s	298MB	273/s	400MB	171ms	284ms	397ms	692ms	917ms
Java 25 (m)	61s	197MB	4.85s	295MB	293/s	396MB	119ms	241ms	383ms	602ms	807ms
Java 17 (g)	66s	189MB	4.824s	271MB	252/s	375MB	188ms	294ms	406ms	701ms	978ms
Java 21 (g)	66s	197MB	4.698s	296MB	278/s	384MB	153ms	282ms	395ms	687ms	932ms
Java 25 (g)	63s	203MB	4.745s	280MB	282/s	386MB	140ms	279ms	390ms	609ms	807ms

Buildpack Jammy Builder Tiny

It's based on Ubuntu Jammy, and works with current JAVA versions. While the base image (default) is bigger the tiny is intended to be used with GraalVM native images.

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <builder>paketobuildpacks/builder-jammy-java-tiny</builder>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	66s	280MB	5.059s	257MB	233/s	371MB	193ms	302ms	477ms	781ms	1048ms
Java 21	65s	307MB	5.323s	275MB	259/s	380MB	181ms	291ms	405ms	701ms	992ms
Java 25	65s	376MB	4.963s	270MB	252/s	448MB	184ms	294ms	408ms	705ms	966ms

Buildpack Custom Distroless Builder

Distroless images are minimal base images that contain only the necessary components to run an application, without any additional packages or tools. This can help reduce the attack surface and improve security.

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <builder>ghcr.io/patbaumgartner/distroless-buildpack-builder:latest</builder>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	97s	298MB	5.236s	260MB	230/s	357MB	194ms	302ms	441ms	764ms	1017ms
Java 21	79s	355MB	4.983s	275MB	257/s	385MB	181ms	292ms	406ms	705ms	994ms
Java 25	75s	424MB	4.981s	270MB	264/s	446MB	181ms	289ms	400ms	683ms	869ms

GraalVM Native Image (CE & Oracle)

A native image is a technology for building Java code into a standalone executable. This executable contains the application classes, classes from its dependencies, runtime library classes and statically linked native code from the JDK. The JVM is packaged in the native image, so there's no need for a Java Runtime Environment on the target system, but the build artifact is platform dependent.

```
mvn -Pnative spring-boot:build-image
```

```
docker run -p 8080:8080 -t spring-petclinic-native:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
CE 25	245s	233MB	0.251s	204MB	1055/s	321MB	45ms	70ms	86ms	113ms	146ms
OG 25	340s	231MB	0.183s	178MB	1260/s	319MB	32ms	60ms	77ms	104ms	140ms

CRaC - OpenJDK (1)

CRaC (Coordinated Restore at Checkpoint) is a feature that allows you to take a snapshot of the state of a Java application and restart it from that state.

Currently only available from:

- Azul Zulu
- Bellsoft Liberica

The application starts within milliseconds!

.. with many other challenges like: embedded configs and secrets, same OS (Linux) and CPU Architecture, same JVM versions, and more ...

CRaC - OpenJDK (2)

```
export JAVA_HOME=/opt/openjdk-17-crac+6_linux-x64/  
export PATH=$JAVA_HOME/bin:$PATH
```

```
mvn clean verify
```

```
java -XX:CRaCCheckpointTo=crac-files -jar target/spring-petclinic-crac-4.0.0-SNAPSHOT.jar
```

```
jcmd target/spring-petclinic-crac-4.0.0-SNAPSHOT.jar JDK.checkpoint
```

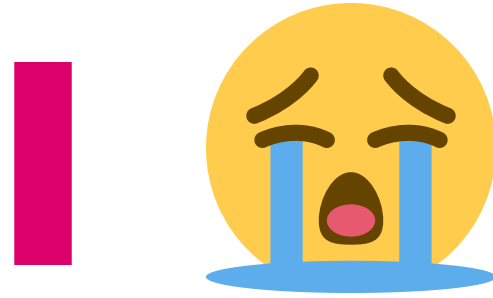
```
java -XX:CRaCRestoreFrom=crac-files
```

CRaC - OpenJDK (3)

CRaC is currently in an experimental state and has the following limitations.

- Since Spring Boot 3.2 CraC support finalised
 - Spring Framework 6.1.0
- Only available for Linux x86 / ARM 64 Bit
- Azul Zulu Build of OpenJDK with CRaC support for development purposes
 - Available for Windows and macOS
 - Simulated checkpoint/restore mechanism for development and testing

Other JVM vendors have similar features, e.g. OpenJ9 with CRIU support.



**No CRaC Buildpacks
for Java & Spring Boot 🍃**

Virtual Threads

A thread is the smallest unit of processing that can be scheduled. It runs concurrently with - and largely independently of - other such units. It is an instance of `java.lang.Thread`. There are two types of threads, platform threads and virtual threads.

```
docker run -e spring.threads.virtual.enabled=true \
-p 8080:8080 -t spring-petclinic-virtual-threads:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
🕒 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 21 🧵	-	-	4.981s	272MB	302/s	370MB	185ms	195ms	201ms	289ms	1496ms
Java 24 🧵	-	-	5.082s	269MB	1721/s	363MB	27ms	30ms	33ms	63ms	355ms
Java 25 🧵	-	-	4.963s	266MB	1713/s	379MB	26ms	29ms	33ms	64ms	940ms
Java 26 🧵	-	-	4.872s	269MB	1728/s	356MB	27ms	31ms	34ms	43ms	64ms

Virtual Threads Java 21

CPU	VT	Startup	Initial RAM	Request/s	RAM	50%	75%	90%	99%	99.9%	99.99%
4 vCPU		4.263s	289MB	1906/s	401MB	21ms	39ms	55ms	81ms	102ms	137ms
4 vCPU		4.631s	290MB	2040/s	400MB	19ms	24ms	42ms	47ms	253ms	615ms
2 vCPU		5.378s	273MB	255/s	377MB	186ms	293ms	405ms	708ms	999ms	1194ms
2 vCPU		4.847s	272MB	312/s	369MB	166ms	193ms	199ms	289ms	1312ms	2003ms
1 vCPU		14.13s	273MB	80/s	367MB	502ms	893ms	1292ms	2198ms	2901ms	3629ms
1 vCPU		14.103s	274MB	110/s	363MB	461ms	498ms	533ms	701ms	2400ms	5527ms
0.5 vCPU		40.16s	271MB	25/s	349MB	1737ms	2735ms	3833ms	6069ms	7916ms	8529ms
0.5 vCPU		40.261s	269MB	37/s	350MB	1396ms	1501ms	1668ms	2402ms	6833ms	7900ms
0.25 vCPU		85.001s	271MB	11/s	340MB	4168ms	6611ms	9303ms	15777ms	19326ms	19326ms
0.25 vCPU		85.136s	271MB	12/s	339MB	4065ms	4462ms	5136ms	8408ms	11641ms	11641ms

Virtual Threads Java 24

CPU	VT	Startup	Initial RAM	Request/s	RAM	50%	75%	90%	99%	99.9%	99.99%
4 vCPU		3.94s	291MB	2050/s	413MB	21ms	34ms	47ms	67ms	83ms	106ms
4 vCPU		3.936s	288MB	2881/s	685MB	11ms	28ms	41ms	76ms	110ms	147ms
2 vCPU		5.08s	272MB	288/s	386MB	138ms	255ms	382ms	604ms	858ms	1048ms
2 vCPU		5.106s	271MB	1722/s	367MB	26ms	30ms	33ms	69ms	428ms	733ms
1 vCPU		15.194s	271MB	83/s	364MB	501ms	837ms	1209ms	2099ms	2934ms	3795ms
1 vCPU		15.053s	269MB	727/s	358MB	58ms	66ms	112ms	181ms	217ms	269ms
0.5 vCPU		43.237s	267MB	25/s	350MB	1734ms	2800ms	3901ms	6118ms	7967ms	9132ms
0.5 vCPU		43.221s	267MB	58/s	351MB	827ms	965ms	1127ms	1638ms	2235ms	2464ms
0.25 vCPU		90.669s	267MB	11/s	327MB	4034ms	6532ms	9566ms	15415ms	20809ms	20809ms
0.25 vCPU		89.97s	265MB	15/s	322MB	3166ms	3634ms	4033ms	7540ms	10232ms	10232ms

Virtual Threads Java 25

CPU	VT	Startup	Initial RAM	Request/s	RAM	50%	75%	90%	99%	99.9%	99.99%
4 vCPU		3.887s	287MB	1924/s	683MB	21ms	38ms	53ms	73ms	88ms	102ms
4 vCPU		3.863s	287MB	2774/s	681MB	5ms	29ms	46ms	85ms	122ms	156ms
2 vCPU		4.901s	270MB	250/s	443MB	187ms	295ms	426ms	675ms	932ms	1117ms
2 vCPU		4.863s	269MB	1757/s	377MB	26ms	28ms	32ms	60ms	937ms	1973ms
1 vCPU		14.072s	270MB	84/s	381MB	500ms	809ms	1233ms	2039ms	2803ms	3728ms
1 vCPU		13.468s	270MB	737/s	355MB	57ms	66ms	111ms	178ms	204ms	263ms
0.5 vCPU		40.636s	266MB	26/s	344MB	1652ms	2667ms	3836ms	6399ms	8938ms	9267ms
0.5 vCPU		41.775s	265MB	60/s	356MB	797ms	903ms	1066ms	1563ms	1667ms	1695ms
0.25 vCPU		86.27s	267MB	11/s	325MB	4034ms	6212ms	8803ms	14479ms	17807ms	17807ms
0.25 vCPU		86.703s	264MB	16/s	324MB	2935ms	3303ms	3634ms	7431ms	14234ms	14234ms

Virtual Threads Java 26

CPU	VT	Startup	Initial RAM	Request/s	RAM	50%	75%	90%	99%	99.9%	99.99%
4 vCPU		3.845s	286MB	1826/s	424MB	23ms	40ms	55ms	75ms	91ms	109ms
4 vCPU		3.797s	285MB	2658/s	438MB	18ms	26ms	40ms	71ms	100ms	127ms
2 vCPU		4.996s	272MB	271/s	388MB	174ms	287ms	398ms	665ms	902ms	1127ms
2 vCPU		4.866s	269MB	1731/s	357MB	27ms	31ms	34ms	44ms	62ms	70ms
1 vCPU		13.319s	271MB	77/s	368MB	560ms	901ms	1307ms	2236ms	2971ms	4040ms
1 vCPU		14.049s	270MB	914/s	352MB	55ms	58ms	62ms	104ms	169ms	186ms
0.5 vCPU		40.558s	267MB	25/s	342MB	1698ms	2677ms	3868ms	6432ms	9100ms	9832ms
0.5 vCPU		41.234s	266MB	64/s	351MB	790ms	897ms	1035ms	1302ms	1499ms	1554ms
0.25 vCPU		86.468s	266MB	11/s	328MB	4200ms	6499ms	9177ms	14391ms	17587ms	17587ms
0.25 vCPU		86.006s	264MB	16/s	319MB	3167ms	3634ms	4129ms	6609ms	7391ms	7391ms

Summary

Summary

- No Optimisations with JDK 17 & JDK 21, ...
- JVM Tuning
- Lazy Spring Beans
- No Spring Boot Actuators
- Fix Spring Boot Config Location
- Disabling JMX
- Dependency Cleanup
- Ahead-of-Time Processing (AOT)
- JLink
- Other JVMs (Eclipse OpenJ9, GraalVM, OpenJDK with CRaC)
- GraalVM Native Image

Conclusions

Conclusions (1)

CPUs

- Your application may not need a full CPU at runtime.
- It will need several CPUs to start as fast as possible (at least 2, 4 is better).
- If you don't mind a slower startup, you can throttle the CPUs below 4.

See: <https://spring.io/blog/2018/11/08/spring-boot-in-a-container>

Conclusions (2)

Request/s

- Every application is different and has different requirements.
- Proper load testing can help find the optimal configuration for your application.

Conclusions (3)

Other Runtimes

- CRIU Support for OpenJDK and OpenJ9 is promising.
 - Supported by Spring since Spring Boot 3.2 / Spring Framework 6.1
- GraalVM Native Image is a great option for Java applications
 - But build times are long
 - The result is different from what you run in your IDE
- Eclipse OpenJ9 is an excellent option for running applications with less memory
 - But startup times are longer than with HotSpot.
- Depending on the distribution, you may get other exciting features.
 - Oracle GraalVM Enterprise Edition, Azul Platform Prime, IBM Semeru Runtime, ...

Conclusions (4)

Other Ideas

- CRaC (Coordinated Restore at Checkpoint)*
- Tree shaking / Obfuscator such as ProGuard*
- AutoConfiguration Filter -> **spring-boot-autoconfiguration-optimizer**
- More JVM tuning (GC, Memory, etc.)
- Project Leyden

A Few Simple Optimisations Applied

```
1  def fib(n):  
2      if n < 2:  
3          return n  
4      return fib(n-1) + fib(n-2)  
5  
6  def fib_memo(n, memo):  
7      if n < 2:  
8          return n  
9      if n in memo:  
10         return memo[n]  
11     memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)  
12     return memo[n]  
13  
14  def fib_memo2(n):  
15      memo = {}  
16      return fib_memo(n, memo)  
17  
18  def fib_iter(n):  
19      a, b = 0, 1  
20      for _ in range(n):  
21          a, b = b, a + b  
22      return a  
23  
24  def fib_iter2(n):  
25      a, b = 0, 1  
26      for _ in range(n):  
27          a, b = b, a + b  
28      return a  
29  
30  def fib_iter3(n):  
31      a, b = 0, 1  
32      for _ in range(n):  
33         a, b = b, a + b  
34     return a  
35  
36  def fib_iter4(n):  
37      a, b = 0, 1  
38      for _ in range(n):  
39         a, b = b, a + b  
40     return a  
41  
42  def fib_iter5(n):  
43      a, b = 0, 1  
44      for _ in range(n):  
45         a, b = b, a + b  
46     return a  
47  
48  def fib_iter6(n):  
49      a, b = 0, 1  
50      for _ in range(n):  
51         a, b = b, a + b  
52     return a  
53  
54  def fib_iter7(n):  
55      a, b = 0, 1  
56      for _ in range(n):  
57         a, b = b, a + b  
58     return a  
59  
60  def fib_iter8(n):  
61      a, b = 0, 1  
62      for _ in range(n):  
63         a, b = b, a + b  
64     return a  
65  
66  def fib_iter9(n):  
67      a, b = 0, 1  
68      for _ in range(n):  
69         a, b = b, a + b  
70     return a  
71  
72  def fib_iter10(n):  
73      a, b = 0, 1  
74      for _ in range(n):  
75         a, b = b, a + b  
76     return a  
77  
78  def fib_iter11(n):  
79      a, b = 0, 1  
80      for _ in range(n):  
81         a, b = b, a + b  
82     return a  
83  
84  def fib_iter12(n):  
85      a, b = 0, 1  
86      for _ in range(n):  
87         a, b = b, a + b  
88     return a  
89  
90  def fib_iter13(n):  
91      a, b = 0, 1  
92      for _ in range(n):  
93         a, b = b, a + b  
94     return a  
95  
96  def fib_iter14(n):  
97      a, b = 0, 1  
98      for _ in range(n):  
99         a, b = b, a + b  
100     return a  
101  
102  def fib_iter15(n):  
103      a, b = 0, 1  
104      for _ in range(n):  
105         a, b = b, a + b  
106     return a  
107  
108  def fib_iter16(n):  
109      a, b = 0, 1  
110      for _ in range(n):  
111         a, b = b, a + b  
112     return a  
113  
114  def fib_iter17(n):  
115      a, b = 0, 1  
116      for _ in range(n):  
117         a, b = b, a + b  
118     return a  
119  
120  def fib_iter18(n):  
121      a, b = 0, 1  
122      for _ in range(n):  
123         a, b = b, a + b  
124     return a  
125  
126  def fib_iter19(n):  
127      a, b = 0, 1  
128      for _ in range(n):  
129         a, b = b, a + b  
130     return a  
131  
132  def fib_iter20(n):  
133      a, b = 0, 1  
134      for _ in range(n):  
135         a, b = b, a + b  
136     return a  
137  
138  def fib_iter21(n):  
139      a, b = 0, 1  
140      for _ in range(n):  
141         a, b = b, a + b  
142     return a  
143  
144  def fib_iter22(n):  
145      a, b = 0, 1  
146      for _ in range(n):  
147         a, b = b, a + b  
148     return a  
149  
150  def fib_iter23(n):  
151      a, b = 0, 1  
152      for _ in range(n):  
153         a, b = b, a + b  
154     return a  
155  
156  def fib_iter24(n):  
157      a, b = 0, 1  
158      for _ in range(n):  
159         a, b = b, a + b  
160     return a  
161  
162  def fib_iter25(n):  
163      a, b = 0, 1  
164      for _ in range(n):  
165         a, b = b, a + b  
166     return a  
167  
168  def fib_iter26(n):  
169      a, b = 0, 1  
170      for _ in range(n):  
171         a, b = b, a + b  
172     return a  
173  
174  def fib_iter27(n):  
175      a, b = 0, 1  
176      for _ in range(n):  
177         a, b = b, a + b  
178     return a  
179  
180  def fib_iter28(n):  
181      a, b = 0, 1  
182      for _ in range(n):  
183         a, b = b, a + b  
184     return a  
185  
186  def fib_iter29(n):  
187      a, b = 0, 1  
188      for _ in range(n):  
189         a, b = b, a + b  
190     return a  
191  
192  def fib_iter30(n):  
193      a, b = 0, 1  
194      for _ in range(n):  
195         a, b = b, a + b  
196     return a  
197  
198  def fib_iter31(n):  
199      a, b = 0, 1  
200      for _ in range(n):  
201         a, b = b, a + b  
202     return a  
203  
204  def fib_iter32(n):  
205      a, b = 0, 1  
206      for _ in range(n):  
207         a, b = b, a + b  
208     return a  
209  
210  def fib_iter33(n):  
211      a, b = 0, 1  
212      for _ in range(n):  
213         a, b = b, a + b  
214     return a  
215  
216  def fib_iter34(n):  
217      a, b = 0, 1  
218      for _ in range(n):  
219         a, b = b, a + b  
220     return a  
221  
222  def fib_iter35(n):  
223      a, b = 0, 1  
224      for _ in range(n):  
225         a, b = b, a + b  
226     return a  
227  
228  def fib_iter36(n):  
229      a, b = 0, 1  
230      for _ in range(n):  
231         a, b = b, a + b  
232     return a  
233  
234  def fib_iter37(n):  
235      a, b = 0, 1  
236      for _ in range(n):  
237         a, b = b, a + b  
238     return a  
239  
240  def fib_iter38(n):  
241      a, b = 0, 1  
242      for _ in range(n):  
243         a, b = b, a + b  
244     return a  
245  
246  def fib_iter39(n):  
247      a, b = 0, 1  
248      for _ in range(n):  
249         a, b = b, a + b  
250     return a  
251  
252  def fib_iter40(n):  
253      a, b = 0, 1  
254      for _ in range(n):  
255         a, b = b, a + b  
256     return a  
257  
258  def fib_iter41(n):  
259      a, b = 0, 1  
260      for _ in range(n):  
261         a, b = b, a + b  
262     return a  
263  
264  def fib_iter42(n):  
265      a, b = 0, 1  
266      for _ in range(n):  
267         a, b = b, a + b  
268     return a  
269  
270  def fib_iter43(n):  
271      a, b = 0, 1  
272      for _ in range(n):  
273         a, b = b, a + b  
274     return a  
275  
276  def fib_iter44(n):  
277      a, b = 0, 1  
278      for _ in range(n):  
279         a, b = b, a + b  
280     return a  
281  
282  def fib_iter45(n):  
283      a, b = 0, 1  
284      for _ in range(n):  
285         a, b = b, a + b  
286     return a  
287  
288  def fib_iter46(n):  
289      a, b = 0, 1  
290      for _ in range(n):  
291         a, b = b, a + b  
292     return a  
293  
294  def fib_iter47(n):  
295      a, b = 0, 1  
296      for _ in range(n):  
297         a, b = b, a + b  
298     return a  
299  
300  def fib_iter48(n):  
301      a, b = 0, 1  
302      for _ in range(n):  
303         a, b = b, a + b  
304     return a  
305  
306  def fib_iter49(n):  
307      a, b = 0, 1  
308      for _ in range(n):  
309         a, b = b, a + b  
310     return a  
311  
312  def fib_iter50(n):  
313      a, b = 0, 1  
314      for _ in range(n):  
315         a, b = b, a + b  
316     return a  
317  
318  def fib_iter51(n):  
319      a, b = 0, 1  
320      for _ in range(n):  
321         a, b = b, a + b  
322     return a  
323  
324  def fib_iter52(n):  
325      a, b = 0, 1  
326      for _ in range(n):  
327         a, b = b, a + b  
328     return a  
329  
330  def fib_iter53(n):  
331      a, b = 0, 1  
332      for _ in range(n):  
333         a, b = b, a + b  
334     return a  
335  
336  def fib_iter54(n):  
337      a, b = 0, 1  
338      for _ in range(n):  
339         a, b = b, a + b  
340     return a  
341  
342  def fib_iter55(n):  
343      a, b = 0, 1  
344      for _ in range(n):  
345         a, b = b, a + b  
346     return a  
347  
348  def fib_iter56(n):  
349      a, b = 0, 1  
350      for _ in range(n):  
351         a, b = b, a + b  
352     return a  
353  
354  def fib_iter57(n):  
355      a, b = 0, 1  
356      for _ in range(n):  
357         a, b = b, a + b  
358     return a  
359  
360  def fib_iter58(n):  
361      a, b = 0, 1  
362      for _ in range(n):  
363         a, b = b, a + b  
364     return a  
365  
366  def fib_iter59(n):  
367      a, b = 0, 1  
368      for _ in range(n):  
369         a, b = b, a + b  
370     return a  
371  
372  def fib_iter60(n):  
373      a, b = 0, 1  
374      for _ in range(n):  
375         a, b = b, a + b  
376     return a  
377  
378  def fib_iter61(n):  
379      a, b = 0, 1  
380      for _ in range(n):  
381         a, b = b, a + b  
382     return a  
383  
384  def fib_iter62(n):  
385      a, b = 0, 1  
386      for _ in range(n):  
387         a, b = b, a + b  
388     return a  
389  
390  def fib_iter63(n):  
391      a, b = 0, 1  
392      for _ in range(n):  
393         a, b = b, a + b  
394     return a  
395  
396  def fib_iter64(n):  
397      a, b = 0, 1  
398      for _ in range(n):  
399         a, b = b, a + b  
400     return a  
401  
402  def fib_iter65(n):  
403      a, b = 0, 1  
404      for _ in range(n):  
405         a, b = b, a + b  
406     return a  
407  
408  def fib_iter66(n):  
409      a, b = 0, 1  
410      for _ in range(n):  
411         a, b = b, a + b  
412     return a  
413  
414  def fib_iter67(n):  
415      a, b = 0, 1  
416      for _ in range(n):  
417         a, b = b, a + b  
418     return a  
419  
420  def fib_iter68(n):  
421      a, b = 0, 1  
422      for _ in range(n):  
423         a, b = b, a + b  
424     return a  
425  
426  def fib_iter69(n):  
427      a, b = 0, 1  
428      for _ in range(n):  
429         a, b = b, a + b  
430     return a  
431  
432  def fib_iter70(n):  
433      a, b = 0, 1  
434      for _ in range(n):  
435         a, b = b, a + b  
436     return a  
437  
438  def fib_iter71(n):  
439      a, b = 0, 1  
440      for _ in range(n):  
441         a, b = b, a + b  
442     return a  
443  
444  def fib_iter72(n):  
445      a, b = 0, 1  
446      for _ in range(n):  
447         a, b = b, a + b  
448     return a  
449  
450  def fib_iter73(n):  
451      a, b = 0, 1  
452      for _ in range(n):  
453         a, b = b, a + b  
454     return a  
455  
456  def fib_iter74(n):  
457      a, b = 0, 1  
458      for _ in range(n):  
459         a, b = b, a + b  
460     return a  
461  
462  def fib_iter75(n):  
463      a, b = 0, 1  
464      for _ in range(n):  
465         a, b = b, a + b  
466     return a  
467  
468  def fib_iter76(n):  
469      a, b = 0, 1  
470      for _ in range(n):  
471         a, b = b, a + b  
472     return a  
473  
474  def fib_iter77(n):  
475      a, b = 0, 1  
476      for _ in range(n):  
477         a, b = b, a + b  
478     return a  
479  
480  def fib_iter78(n):  
481      a, b = 0, 1  
482      for _ in range(n):  
483         a, b = b, a + b  
484     return a  
485  
486  def fib_iter79(n):  
487      a, b = 0, 1  
488      for _ in range(n):  
489         a, b = b, a + b  
490     return a  
491  
492  def fib_iter80(n):  
493      a, b = 0, 1  
494      for _ in range(n):  
495         a, b = b, a + b  
496     return a  
497  
498  def fib_iter81(n):  
499      a, b = 0, 1  
500      for _ in range(n):  
501         a, b = b, a + b  
502     return a  
503  
504  def fib_iter82(n):  
505      a, b = 0, 1  
506      for _ in range(n):  
507         a, b = b, a + b  
508     return a  
509  
510  def fib_iter83(n):  
511      a, b = 0, 1  
512      for _ in range(n):  
513         a, b = b, a + b  
514     return a  
515  
516  def fib_iter84(n):  
517      a, b = 0, 1  
518      for _ in range(n):  
519         a, b = b, a + b  
520     return a  
521  
522  def fib_iter85(n):  
523      a, b = 0, 1  
524      for _ in range(n):  
525         a, b = b, a + b  
526     return a  
527  
528  def fib_iter86(n):  
529      a, b = 0, 1  
530      for _ in range(n):  
531         a, b = b, a + b  
532     return a  
533  
534  def fib_iter87(n):  
535      a, b = 0, 1  
536      for _ in range(n):  
537         a, b = b, a + b  
538     return a  
539  
540  def fib_iter88(n):  
541      a, b = 0, 1  
542      for _ in range(n):  
543         a, b = b, a + b  
544     return a  
545  
546  def fib_iter89(n):  
547      a, b = 0, 1  
548      for _ in range(n):  
549         a, b = b, a + b  
550     return a  
551  
552  def fib_iter90(n):  
553      a, b = 0, 1  
554      for _ in range(n):  
555         a, b = b, a + b  
556     return a  
557  
558  def fib_iter91(n):  
559      a, b = 0, 1  
560      for _ in range(n):  
561         a, b = b, a + b  
562     return a  
563  
564  def fib_iter92(n):  
565      a, b = 0, 1  
566      for _ in range(n):  
567         a, b = b, a + b  
568     return a  
569  
570  def fib_iter93(n):  
571      a, b = 0, 1  
572      for _ in range(n):  
573         a, b = b, a + b  
574     return a  
575  
576  def fib_iter94(n):  
577      a, b = 0, 1  
578      for _ in range(n):  
579         a, b = b, a + b  
580     return a  
581  
582  def fib_iter95(n):  
583      a, b = 0, 1  
584      for _ in range(n):  
585         a, b = b, a + b  
586     return a  
587  
588  def fib_iter96(n):  
589      a, b = 0, 1  
590      for _ in range(n):  
591         a, b = b, a + b  
592     return a  
593  
594  def fib_iter97(n):  
595      a, b = 0, 1  
596      for _ in range(n):  
597         a, b = b, a + b  
598     return a  
599  
600  def fib_iter98(n):  
601      a, b = 0, 1  
602      for _ in range(n):  
603         a, b = b, a + b  
604     return a  
605  
606  def fib_iter99(n):  
607      a, b = 0, 1  
608      for _ in range(n):  
609         a, b = b, a + b  
610     return a  
611  
612  def fib_iter100(n):  
613      a, b = 0, 1  
614      for _ in range(n):  
615         a, b = b, a + b  
616     return a  
617  
618  def fib_iter101(n):  
619      a, b = 0, 1  
620      for _ in range(n):  
621         a, b = b, a + b  
622     return a  
623  
624  def fib_iter102(n):  
625      a, b = 0, 1  
626      for _ in range(n):  
627         a, b = b, a + b  
628     return a  
629  
630  def fib_iter103(n):  
631      a, b = 0, 1  
632      for _ in range(n):  
633         a, b = b, a + b  
634     return a  
635  
636  def fib_iter104(n):  
637      a, b = 0, 1  
638      for _ in range(n):  
639         a, b = b, a + b  
640     return a  
641  
642  def fib_iter105(n):  
643      a, b = 0, 1  
644      for _ in range(n):  
645         a, b = b, a + b  
646     return a  
647  
648  def fib_iter106(n):  
649      a, b = 0, 1  
650      for _ in range(n):  
651         a, b = b, a + b  
652     return a  
653  
654  def fib_iter107(n):  
655      a, b = 0, 1  
656      for _ in range(n):  
657         a, b = b, a + b  
658     return a  
659  
660  def fib_iter108(n):  
661      a, b = 0, 1  
662      for _ in range(n):  
663         a, b = b, a + b  
664     return a  
665  
666  def fib_iter109(n):  
667      a, b = 0, 1  
668      for _ in range(n):  
669         a, b = b, a + b  
670     return a  
671  
672  def fib_iter110(n):  
673      a, b = 0, 1  
674      for _ in range(n):  
675         a, b = b, a + b  
676     return a  
677  
678  def fib_iter111(n):  
679      a, b = 0, 1  
680      for _ in range(n):  
681         a, b = b, a + b  
682     return a  
683  
684  def fib_iter112(n):  
685      a, b = 0, 1  
686      for _ in range(n):  
687         a, b = b, a + b  
688     return a  
689  
690  def fib_iter113(n):  
691      a, b = 0, 1  
692      for _ in range(n):  
693         a, b = b, a + b  
694     return a  
695  
696  def fib_iter114(n):  
697      a, b = 0, 1  
698      for _ in range(n):  
699         a, b = b, a + b  
700     return a  
701  
702  def fib_iter115(n):  
703      a, b = 0, 1  
704      for _ in range(n):  
705         a, b = b, a + b  
706     return a  
707  
708  def fib_iter116(n):  
709      a, b = 0, 1  
710      for _ in range(n):  
711         a, b = b, a + b  
712     return a  
713  
714  def fib_iter117(n):  
715      a, b = 0, 1  
716      for _ in range(n):  
717         a, b = b, a + b  
718     return a  
719  
720  def fib_iter118(n):  
721      a, b = 0, 1  
722      for _ in range(n):  
723         a, b = b, a + b  
724     return a  
725  
726  def fib_iter119(n):  
727      a, b = 0, 1  
728      for _ in range(n):  
729         a, b = b, a + b  
730     return a  
731  
732  def fib_iter120(n):  
733      a, b = 0, 1  
734      for _ in range(n):  
735         a, b = b, a + b  
736     return a  
737  
738  def fib_iter121(n):  
739      a, b = 0, 1  
740      for _ in range(n):  
741         a, b = b, a + b  
742     return a  
743  
744  def fib_iter122(n):  
745      a, b = 0, 1  
746      for _ in range(n):  
747         a, b = b, a + b  
748     return a  
749  
750  def fib_iter123(n):  
751      a, b = 0, 1  
752      for _ in range(n):  
753         a, b = b, a + b  
754     return a  
755  
756  def fib_iter124(n):  
757      a, b = 0, 1  
758      for _ in range(n):  
759         a, b = b, a + b  
760     return a  
761  
762  def fib_iter125(n):  
763      a, b = 0, 1  
764      for _ in range(n):  
765         a, b = b, a + b  
766     return a  
767  
768  def fib_iter126(n):  
769      a, b = 0, 1  
770      for _ in range(n):  
771         a, b = b, a + b  
772     return a  
773  
774  def fib_iter127(n):  
775      a, b = 0, 1  
776      for _ in range(n):  
777         a, b = b, a + b  
778     return a  
779  
780  def fib_iter128(n):  
781      a, b = 0, 1  
782      for _ in range(n):  
783         a, b = b, a + b  
784     return a  
785  
786  def fib_iter129(n):  
787      a, b = 0, 1  
788      for _ in range(n):  
789         a, b = b, a + b  
790     return a  
791  
792  def fib_iter130(n):  
793      a, b = 0, 1  
794      for _ in range(n):  
795         a, b = b, a + b  
796     return a  
797  
798  def fib_iter131(n):  
799      a, b = 0, 1  
800      for _ in range(n):  
801         a, b = b, a + b  
802     return a  
803  
804  def fib_iter132(n):  
805      a, b = 0, 1  
806      for _ in range(n):  
807         a, b = b, a + b  
808     return a  
809  
810  def fib_iter133(n):  
811      a, b = 0, 1  
812      for _ in range(n):  
813         a, b = b, a + b  
814     return a  
815  
816  def fib_iter134(n):  
817      a, b = 0, 1  
818      for _ in range(n):  
819         a, b = b, a + b  
820     return a  
821  
822  def fib_iter135(n):  
823      a, b = 0, 1  
824      for _ in range(n):  
825         a, b = b, a + b  
826     return a  
827  
828  def fib_iter136(n):  
829      a, b = 0, 1  
830      for _ in range(n):  
831         a, b = b, a + b  
832     return a  
833  
834  def fib_iter137(n):  
835      a, b = 0, 1  
836      for _ in range(n):  
837         a, b = b, a + b  
838     return a  
839  
840  def fib_iter138(n):  
841      a, b = 0, 1  
842      for _ in range(n):  
843         a, b = b, a + b  
844     return a  
845  
846  def fib_iter139(n):  
847      a, b = 0, 1  
848      for _ in range(n):  
849         a, b = b, a + b  
850     return a  
851  
852  def fib_iter140(n):  
853      a, b = 0, 1  
854      for _ in range(n):  
855         a, b = b, a + b  
856     return a  
857  
858  def fib_iter141(n):  
859      a, b = 0, 1  
860      for _ in range(n):  
861         a, b = b, a + b  
862     return a  
863  
864  def fib_iter142(n):  
865      a, b = 0, 1  
866      for _ in range(n):  
867         a, b = b, a + b  
868     return a  
869  
870  def fib_iter143(n):  
871      a, b = 0, 1  
872      for _ in range(n):  
873         a, b = b, a + b  
874     return a  
875  
876  def fib_iter144(n):  
877      a, b = 0, 1  
878      for _ in range(n):  
879         a, b = b, a + b  
880     return a  
881  
882  def fib_iter145(n):  
883      a, b = 0, 1  
884      for _ in range(n):  
885         a, b = b, a + b  
886     return a  
887  
888  def fib_iter146(n):  
889      a, b = 0, 1  
890      for _ in range(n):  
891         a, b = b, a + b  
892     return a  
893  
894  def fib_iter147(n):  
895      a, b = 0, 1  
896      for _ in range(n):  
897         a, b = b, a + b  
898     return a  
899  
900  def fib_iter148(n):  
901      a, b = 0, 1  
902      for _ in range(n):  
903         a, b = b, a + b  
904     return a  
905  
906  def fib_iter149(n):  
907      a, b = 0, 1  
908      for _ in range(n):  
909         a, b = b, a + b  
910     return a  
911  
912  def fib_iter150(n):  
913      a, b = 0, 1  
914      for _ in range(n):  
915         a, b = b, a + b  
916     return a  
917  
918  def fib_iter151(n):  
919      a, b = 0, 1  
920      for _ in range(n):  
921         a, b = b, a + b  
922     return a  
923  
924  def fib_iter152(n):  
925      a, b = 0, 1  
926      for _ in range(n):  
927         a, b = b, a + b  
928     return a  
929  
930  def fib_iter153(n):  
931      a, b = 0, 1  
932      for _ in range(n):  
933         a, b = b, a + b  
934     return a  
935  
936  def fib_iter154(n):  
937      a, b = 0, 1  
938      for _ in range(n):  
939         a, b = b, a + b  
940     return a  
941  
942  def fib_iter155(n):  
943      a, b = 0, 1  
944      for _ in range(n):  
945         a, b = b, a + b  
946     return a  
947  
948  def fib_iter156(n):  
949      a, b = 0, 1  
950      for _ in range(n):  
951         a, b = b, a + b  
952     return a  
953  
954  def fib_iter157(n):  
955      a, b = 0, 1  
956      for _ in range(n):  
957         a, b = b, a + b  
958     return a  
959  
960  def fib_iter158(n):  
961      a, b = 0, 1  
962      for _ in range(n):  
963         a, b = b, a + b  
964     return a  
965  
966  def fib_iter159(n):  
967      a, b = 0, 1  
968      for _ in range(n):  
969         a, b = b, a + b  
970     return a  
971  
972  def fib_iter160(n):  
973      a, b = 0, 1  
974      for _ in range(n):  
975         a, b = b, a + b  
976     return a  
977  
978  def fib_iter161(n):  
979      a, b = 0, 1  
980      for _ in range(n):  
981         a, b = b, a + b  
982     return a  
983  
984  def fib_iter162(n):  
985      a, b = 0, 1  
986      for _ in range(n):  
987         a, b = b, a + b  
988     return a  
989  
990  def fib_iter163(n):  
991      a, b = 0, 1  
992      for _ in range(n):  
993         a, b = b, a + b  
994     return a  
995  
996  def fib_iter164(n):  
997      a, b = 0, 1  
998      for _ in range(n):  
999         a, b = b, a + b  
1000     return a  
1001  
1002  def fib_iter165(n):  
1003      a, b = 0, 1  
1004      for _ in range(n):  
1005         a, b = b, a + b  
1006     return a  
1007  
1008  def fib_iter166(n):  
1009      a, b = 0, 1  
1010      for _ in range(n):  
1011         a, b = b, a + b  
1012     return a  
1013  
1014  def fib_iter167(n):  
1015      a, b = 0, 1  
1016      for _ in range(n):  
1017         a, b = b, a + b  
1018     return a  
1019  
1020  def fib_iter168(n):  
1021      a, b = 0, 1  
1022      for _ in range(n):  
1023         a, b = b, a + b  
1024     return a  
1025  
1026  def fib_iter169(n):  
1027      a, b = 0, 1  
1028      for _ in range(n):  
1029         a, b = b, a + b  
1030     return a  
1031  
1032  def fib_iter170(n):  
1033      a, b = 0, 1  
1034      for _ in range(n):  
1035         a, b = b, a + b  
1036     return a  
1037  
1038  def fib_iter171(n):  
1039      a, b = 0, 1  
1040      for _ in range(n):  
1041         a, b = b, a + b  
1042     return a  
1043  
1044  def fib_iter172(n):  
1045      a, b = 0, 1  
1046      for _ in range(n):  
1047         a, b = b, a + b  
1048     return a  
1049  
1050  def fib_iter173(n):  
1051      a, b = 0, 1  
1052      for _ in range(n):  
1053         a, b = b, a + b  
1054     return a  

```

A Few Simple Optimisations Applied

- Dependency Cleanup
 - DB Drivers, Spring Boot Actuator, Jackson, Tomcat Websocket, ...
- Bellsoft Builder (musl) / Base Builder Tiny
- JLink
- JVM Parameters (java-memory-calculator)
- Application Class Data Sharing (AppCDS)
- Spring AOT
- Lazy Spring Beans
- Fixing Spring Boot Config Location
- Virtual Threads

Bellsoft Builder (musl)

```
sdk use java 21.0.10-librca

mvn spring-boot:build-image

docker run -p 8080:8080 \
  -e spring.threads.virtual.enabled=true \
  -e spring.main.lazy-initialization=true \
  -e spring.data.jpa.repositories.bootstrap-mode=lazy \
  -e spring.config.location=classpath:application.properties \
  -t spring-petclinic-optimized-builder-bellsoft-musl:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
 Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	48s	145MB	3.348s	224MB	263/s	368MB	126ms	290ms	401ms	704ms	995ms
Java 21 	51s	211MB	2.918s	228MB	361/s	370MB	101ms	200ms	301ms	540ms	785ms
Java 25 	57s	318MB	3.003s	228MB	433/s	670MB	97ms	186ms	278ms	463ms	610ms

Base Builder Tiny

```
docker run -p 8080:8080 \
  -e spring.threads.virtual.enabled=true \
  -e spring.main.lazy-initialization=true \
  -e spring.data.jpa.repositories.bootstrap-mode=lazy \
  -e spring.config.location=classpath:application.properties \
  -t spring-petclinic-optimized-builder-tiny-virtual-threads:4.0.0-SNAPSHOT
```

	VT	AC	COH	Build	Image	Startup	In. RAM	R/s	RAM	50%	75%	90%	99%	99.9%
 Java 17				66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17				60s	311MB	3.14s	216MB	324/s	346MB	105ms	209ms	333ms	602ms	885ms
Java 21				58s	318MB	2.921s	222MB	346/s	360MB	103ms	204ms	308ms	589ms	798ms
Java 25				61s	344MB	3.079s	221MB	438/s	645MB	96ms	185ms	279ms	483ms	657ms
Java 25				62s	355MB	3.829s	252MB	349/s	525MB	107ms	203ms	304ms	513ms	702ms
Java 25				62s	355MB	3.383s	233MB	2256/s	350MB	21ms	22ms	25ms	36ms	651ms
Java 25				58s	547MB	3.927s	241MB	367/s	519MB	105ms	199ms	297ms	501ms	694ms
Java 25				58s	547MB	3.394s	227MB	2256/s	337MB	20ms	22ms	25ms	36ms	585ms

Did I miss something? 🧐

Let me/us know! 🙋

... or get in touch later! 🙈

Feedback is a Gift

Your turn 🙋

👉 Use the Devvix Companion App

- 🐛 What didn't work (aka bugs)?
- 🤖 What was confusing (missing docs)?
- 🚀 What should I refactor next?

No feedback = it works on your machine™



Spring Boot in the Cloud

Advanced Optimization Deep Dive

Patrick Baumgartner

42talents GmbH, Zürich, Switzerland

@patbaumgartner

patrick.baumgartner@42talents.com

bit.ly/48i5U9x

